10 / SLIDE 001 Testing del software e delle API

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 1 di 14

Spiegazione per studiare

Il testing verifica che un programma e le sue interfacce si comportino come previsto. La lezione copre strategie di test, TDD, coverage, mocking, test parametrizzati e verifiche di endpoint Web con FastAPI.

Definizioni, passaggi ed esempi presenti nella slide

Testing

SLIDE ORIGINALE · diagrammi, tabelle e codice





Testing

10 / SLIDE 002 Introduzione al testing

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 2 di 14

Spiegazione per studiare

Testare riduce bug e regressioni e rende più sicuro modificare il codice. Gli unit test controllano unità isolate, i test di integrazione la collaborazione fra componenti, gli end-to-end il flusso visto dall'utente e i regression test il mantenimento di funzionalità precedenti.

Definizioni, passaggi ed esempi presenti nella slide

Perché testare il codice?

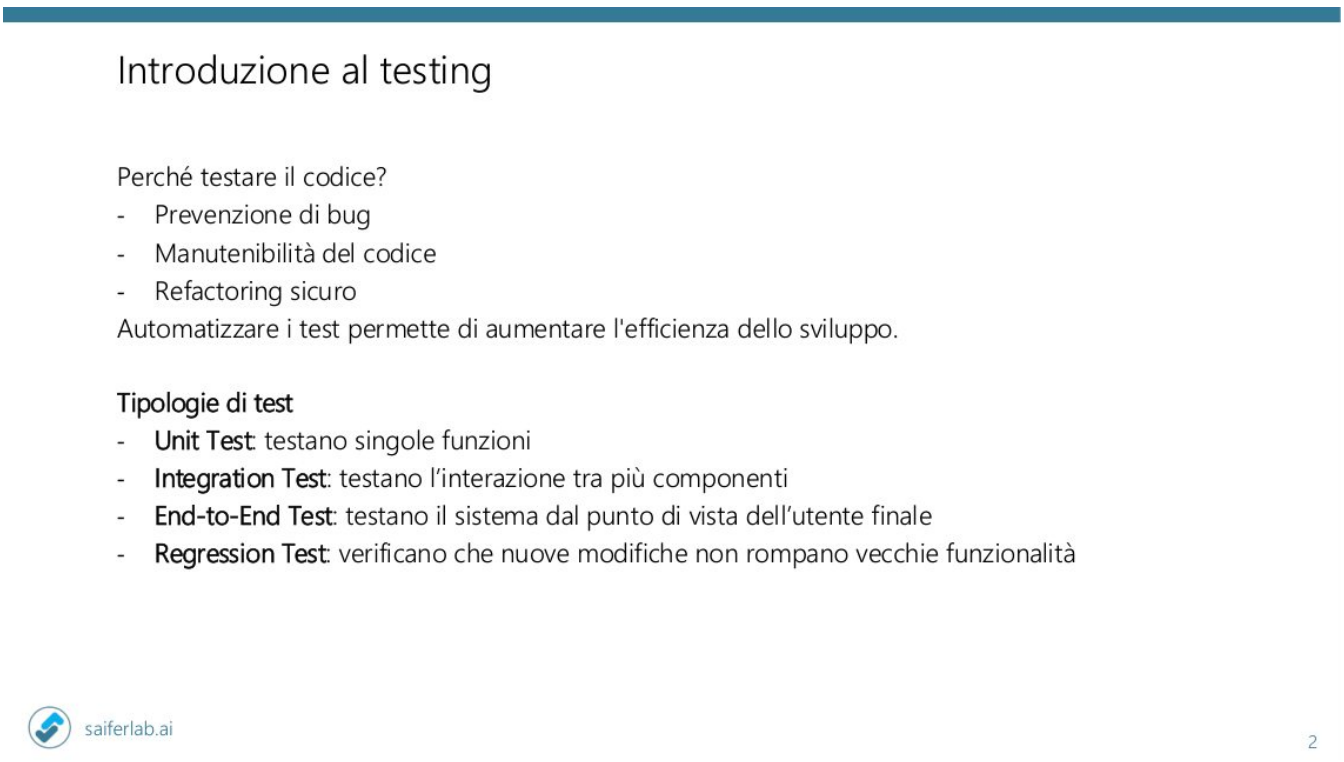
- Prevenzione di bug
- Manutenibilità del codice
- Refactoring sicuro

Automatizzare i test permette di aumentare l'efficienza dello sviluppo.

Tipologie di test

- Unit Test: testano singole funzioni
- Integration Test: testano l’interazione tra più componenti
- End-to-End Test: testano il sistema dal punto di vista dell’utente finale
- Regression Test: verificano che nuove modifiche non rompano vecchie funzionalità

SLIDE ORIGINALE · diagrammi, tabelle e codice



10 / SLIDE 003 Test-Driven Development (TDD)

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 3 di 14

Spiegazione per studiare

Nel Test-Driven Development si scrive prima un test che fallisce, poi il codice minimo per farlo passare e infine si riorganizza il codice. Il ciclo aiuta a chiarire il comportamento atteso e può favorire un'architettura più modulare.

Definizioni, passaggi ed esempi presenti nella slide

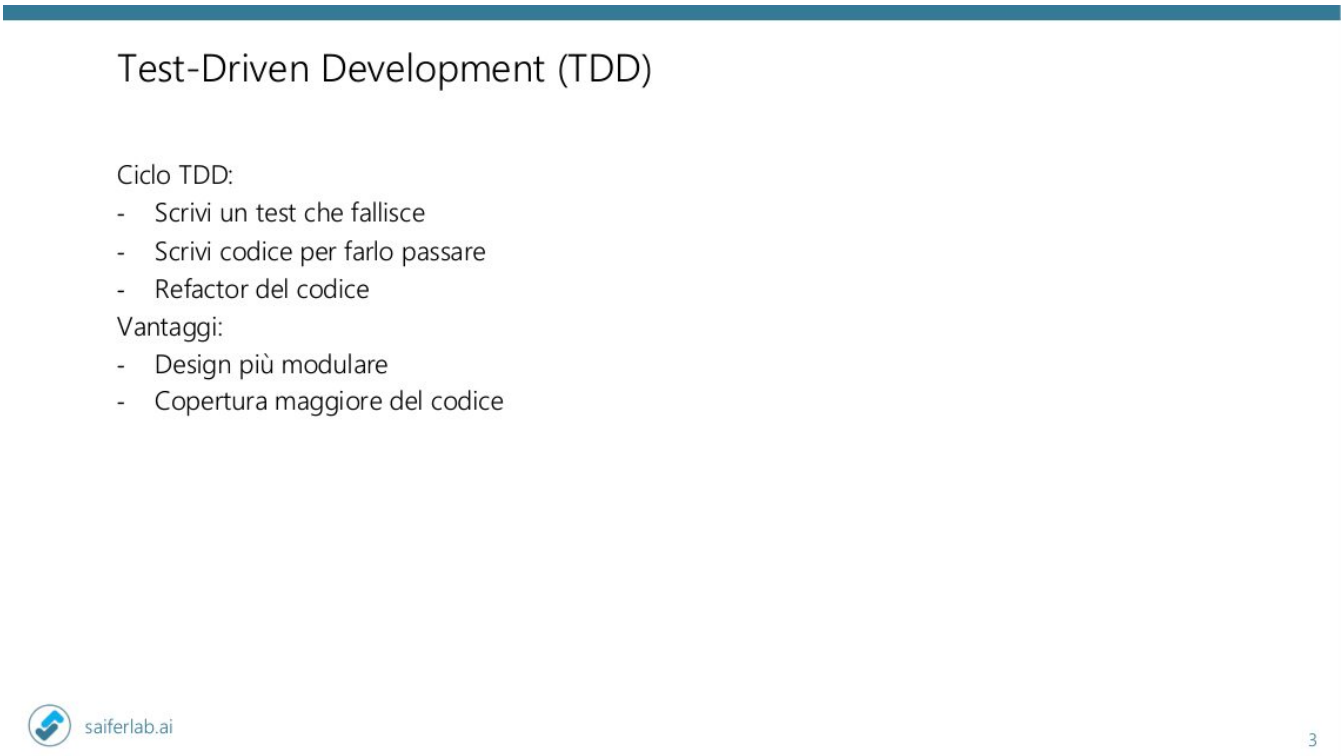
Ciclo TDD:

- Scrivi un test che fallisce
- Scrivi codice per farlo passare
- Refactor del codice

Vantaggi:

- Design più modulare
- Copertura maggiore del codice

SLIDE ORIGINALE · diagrammi, tabelle e codice



10 / SLIDE 004 Esercizi di test semplici in Python

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 4 di 14

Spiegazione per studiare

L'esempio con `pytest` definisce una funzione di somma e verifica il risultato mediante `assert`. Il caso di test confronta l'output calcolato con quello previsto: se la condizione è falsa, il framework segnala il fallimento. Gli esercizi suggeriscono ulteriori funzioni da testare.

Definizioni, passaggi ed esempi presenti nella slide

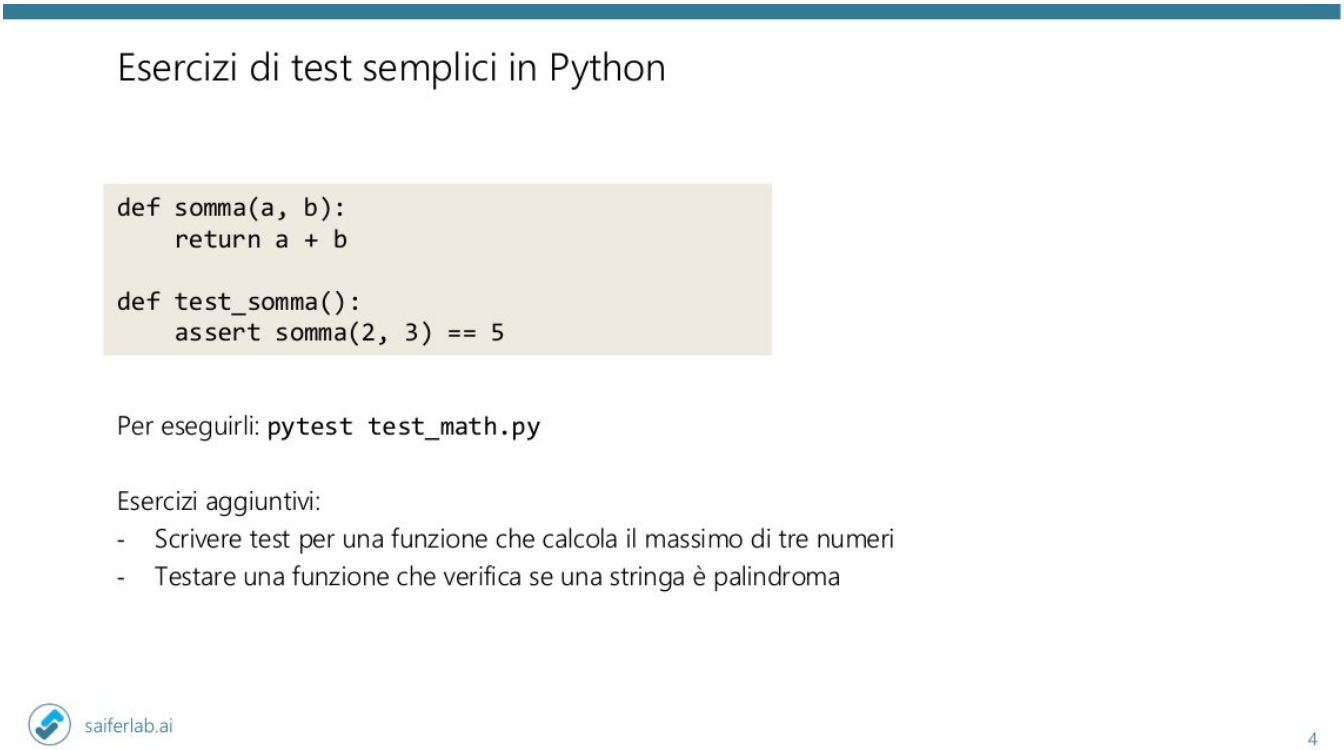
```
def somma(a, b):
return a + b
def test_somma():
assert somma(2, 3) == 5
```

Per eseguirli: pytest test_math.py

Esercizi aggiuntivi:

- Scrivere test per una funzione che calcola il massimo di tre numeri
- Testare una funzione che verifica se una stringa è palindroma

SLIDE ORIGINALE · diagrammi, tabelle e codice



10 / SLIDE 005 Code Coverage

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 5 di 14

Spiegazione per studiare

Il code coverage misura quanta parte del codice viene eseguita durante una suite di test. Il valore non dimostra da solo che siano stati verificati tutti i comportamenti corretti: una riga eseguita può contenere ancora un errore non individuato.

Definizioni, passaggi ed esempi presenti nella slide

Il code coverage è la percentuale di codice eseguito durante i test

```
def somma_pari(a, b):
if a % 2 == 0 and b % 2 == 0:
return a + b # linea non testata
return None
def test_somma():
assert somma(2, 3) == 5
```

Attenzione: alta coverage ≠test efficaci

SLIDE ORIGINALE · diagrammi, tabelle e codice

Code Coverage

Il code coverage è la percentuale di codice eseguito durante i test

```
def somma_pari(a, b):
    if a % 2 == 0 and b % 2 == 0:
        return a + b # linea non testata
    return None

def test_somma():
    assert somma(2, 3) == 5
```

Attenzione: **alta coverage ≠ test efficaci**

saiferlab.ai

5

10 / SLIDE 006 Mocking e isolamento

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 6 di 14

Spiegazione per studiare

Il mocking sostituisce temporaneamente una dipendenza reale con un oggetto controllabile nel test. È utile per isolare una funzione da database, rete o file system, rendendo la verifica ripetibile e indipendente dai servizi esterni.

Definizioni, passaggi ed esempi presenti nella slide

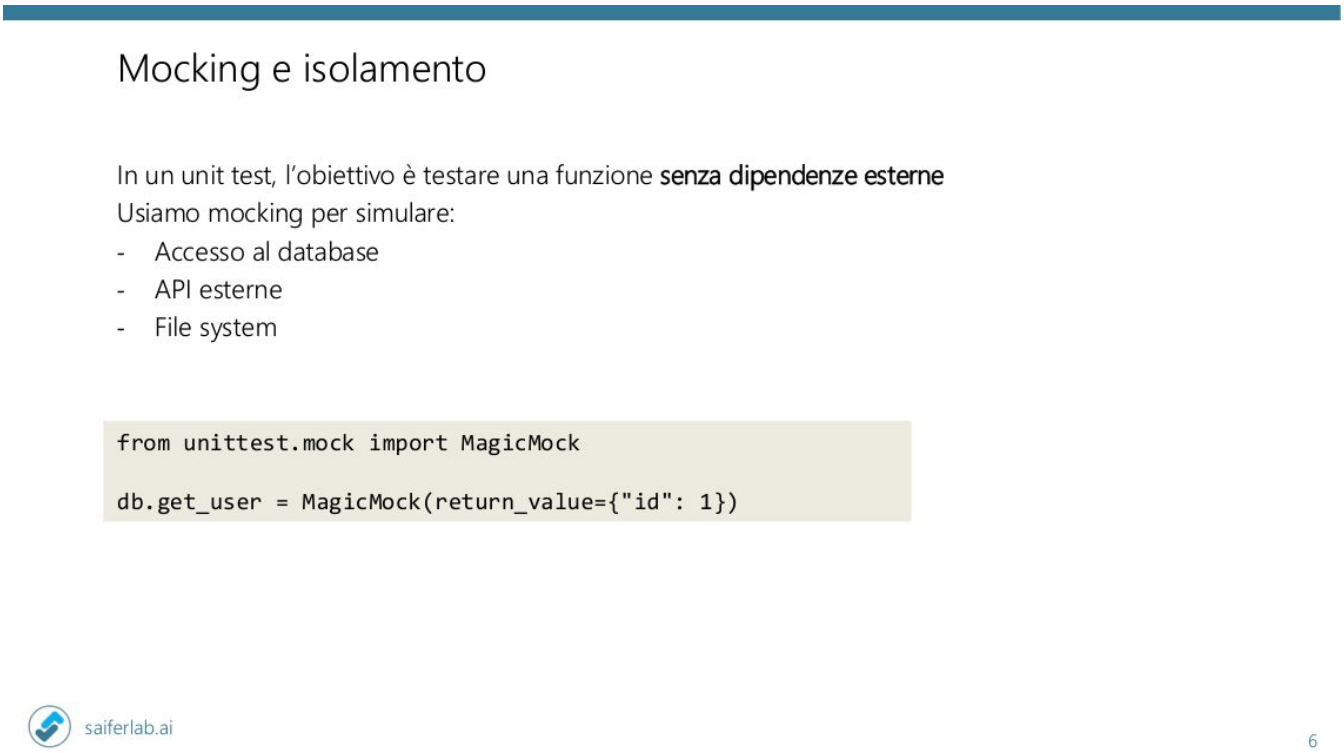
In un unit test, l'obiettivo è testare una funzione senza dipendenze esterne Usiamo mocking per simulare:

- Accesso al database
- API esterne
- File system

```
from unittest.mock import MagicMock
```

db.get_user = MagicMock(return_value={"id": 1})

SLIDE ORIGINALE · diagrammi, tabelle e codice



10 / SLIDE 007 Tutorial di mocking

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 7 di 14

Spiegazione per studiare

Nel tutorial la funzione da verificare dipende da un metodo di accesso al database non implementato. L'obiettivo dell'unit test è controllare il calcolo della somma senza dover costruire anche il database: la dipendenza verrà rimpiazzata con un mock.

Definizioni, passaggi ed esempi presenti nella slide

Per esempio, si può utilizzare il mocking per implementare un metodo che dipende da componenti esterni Per lo scopo dello unit test infatti, non serve testare anche i metodi esterni

In questo caso, non ci preoccuperemo di implementare la funzione che accede al database, ma vedremo che è comunque possibile fare il test di una funzione che dipende da un medodo non implementato

```
class Database:
def get_valori(self):
def somma(db: Database):
a, b = db.get_valori()
return a + b
```

SLIDE ORIGINALE · diagrammi, tabelle e codice

Tutorial di mocking

Per esempio, si può utilizzare il mocking per implementare un metodo che dipende da componenti esterni
Per lo scopo dello unit test infatti, non serve testare anche i metodi esterni

In questo caso, non ci preoccuperemo di implementare la funzione che accede al database, ma vedremo che è comunque possibile fare il test di una funzione che dipende da un medodo non implementato

```
class Database:
    def get_valori(self):
        # per ora non lo implementiamo
        raise NotImplementedError


def somma(db: Database):
    a, b = db.get_valori()
    return a + b
```

saiferlab.ai

7

10 / SLIDE 008 Tutorial di mocking

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 8 di 14

Spiegazione per studiare

`MagicMock` simula il comportamento del database e fa restituire una coppia di valori prestabilita. Il test chiama la funzione e confronta il risultato con l'atteso. Questo esempio mostra come controllare il confine tra codice sotto test e componente esterno.

Definizioni, passaggi ed esempi presenti nella slide

```
import unittest
from unittest.mock import MagicMock
from somma_da_db import somma
```

Implementiamo un metodo mock che finge di chiamare il metodo non implementato del databse e restituisce dei valori fittizi

```
class TestConMockDatabase(unittest.TestCase):
```

All'interno del test potremo chiamare il metodo senza avere un errore.

```
def test_somma_mock_db(self):
```

risultato = somma(mock_db)

self.assertEqual(risultato, 8)

```
if __name__ == "__main__":
```

unittest.main()

SLIDE ORIGINALE · diagrammi, tabelle e codice

Tutorial di mocking

Implementiamo un metodo mock che finge di chiamare il metodo non implementato del databse e restituisce dei valori fittizi

All'interno del test potremo chiamare il metodo senza avere un errore.

```
import unittest
from unittest.mock import MagicMock
from somma_da_db import somma


class TestConMockDatabase(unittest.TestCase):
     def test_somma_mock_db(self):
          # Crea un mock del database
          mock_db = MagicMock()
          mock_db.get_valori.return_value = (3, 5)

          risultato = somma(mock_db)

          self.assertEqual(risultato, 8)

if __name__ == "__main__":
     unittest.main()
```

saiferlab.ai

8

10 / SLIDE 009 Test parametrici

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 9 di 14

Spiegazione per studiare

La parametrizzazione di `pytest` usa lo stesso test con input e valori attesi differenti. Riduce ripetizioni e permette di coprire più casi, inclusi numeri negativi e valori limite, mantenendo una sola definizione del comportamento verificato.

Definizioni, passaggi ed esempi presenti nella slide

Si possono scrivere test che sfruttano un’interfaccia comune ma usano parametri diversi Evita codice ripetuto, migliora la copertura

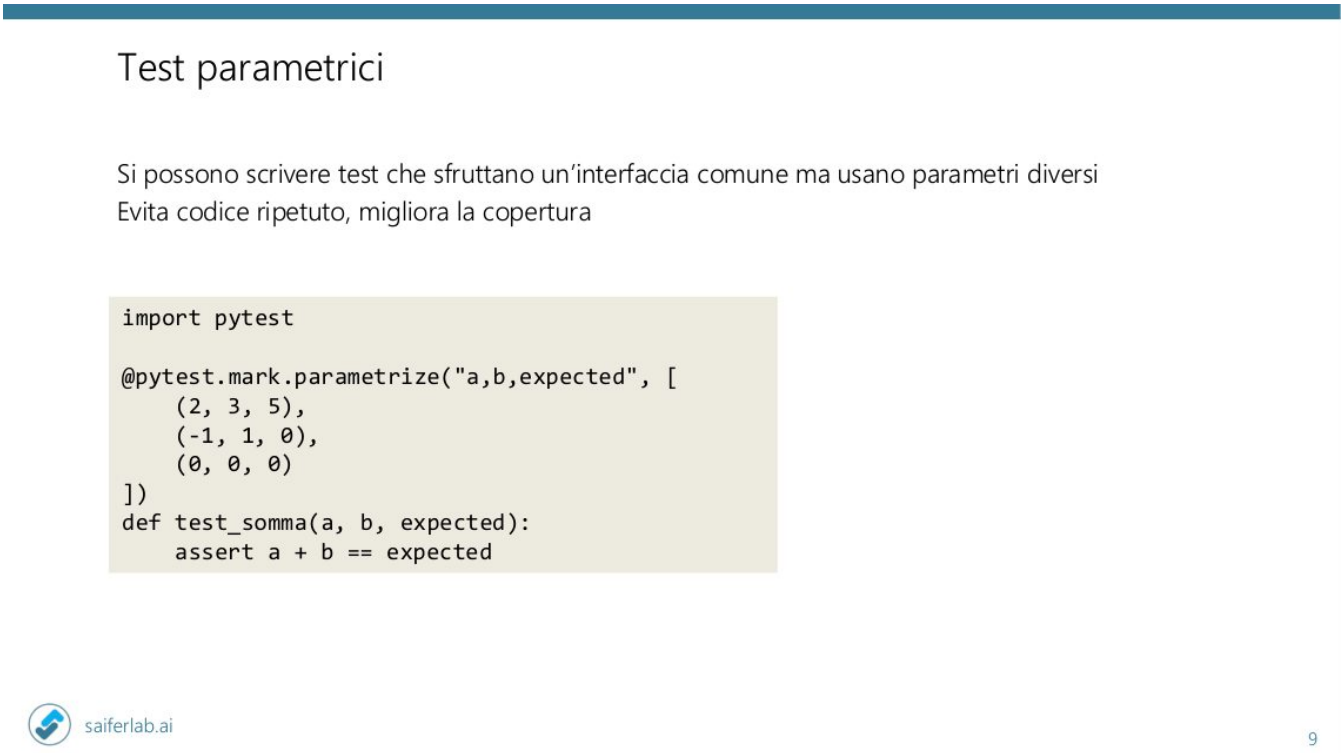
```
import pytest

@pytest.mark.parametrize("a,b,expected", [

(2, 3, 5),
(-1, 1, 0),
(0, 0, 0)
])
def test_somma(a, b, expected):

assert a + b == expected
```

SLIDE ORIGINALE · diagrammi, tabelle e codice



10 / SLIDE 010 Testing per il Web Programming

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 10 di 14

Spiegazione per studiare

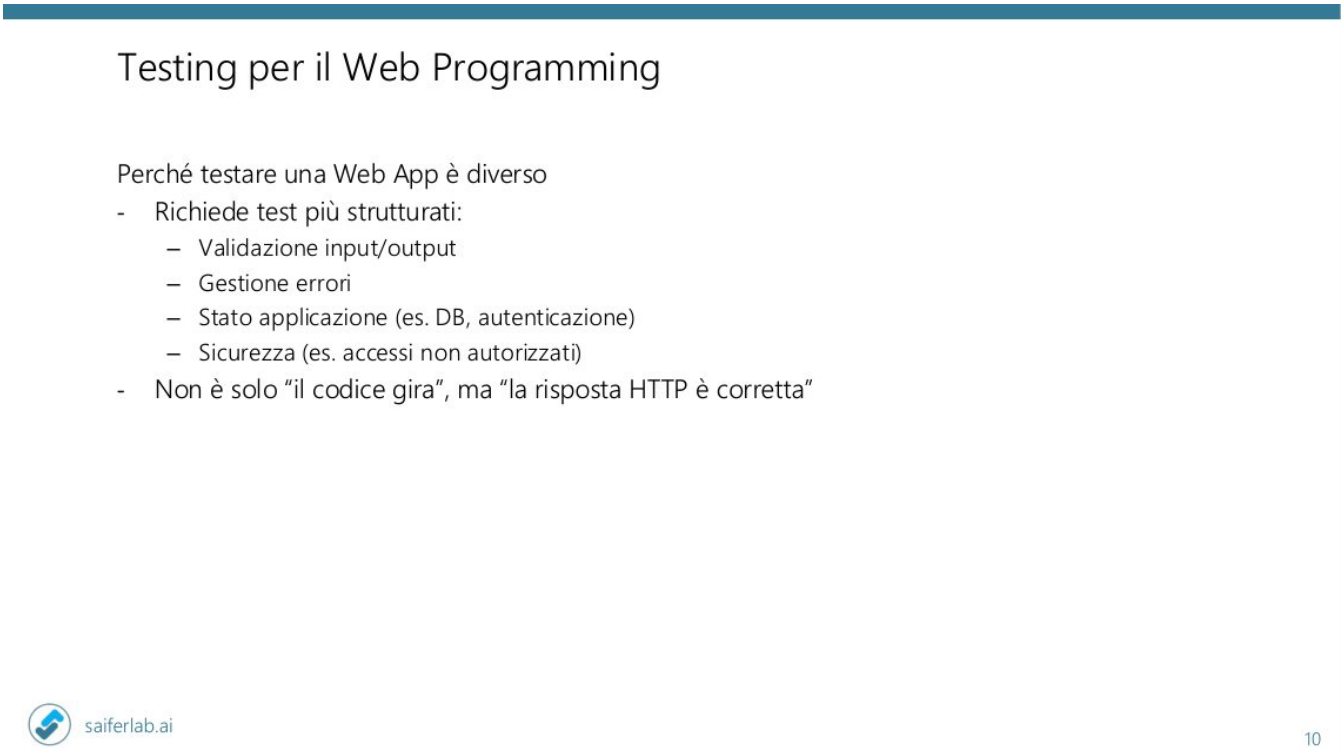
Il testing di una Web app deve considerare anche il protocollo: input, risposta HTTP, stato dell'applicazione, autenticazione e gestione degli errori. Non basta che la funzione Python termini correttamente: deve produrre status code e dati coerenti con l'interfaccia pubblica.

Definizioni, passaggi ed esempi presenti nella slide

Perché testare una Web App è diverso

- Richiede test più strutturati:
- Validazione input/output
- Gestione errori
- Stato applicazione (es. DB, autenticazione)
- Sicurezza (es. accessi non autorizzati)
- Non è solo “il codice gira”, ma “la risposta HTTP è corretta”

SLIDE ORIGINALE · diagrammi, tabelle e codice



10 / SLIDE 011 Casi di fallimento da testare

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 11 di 14

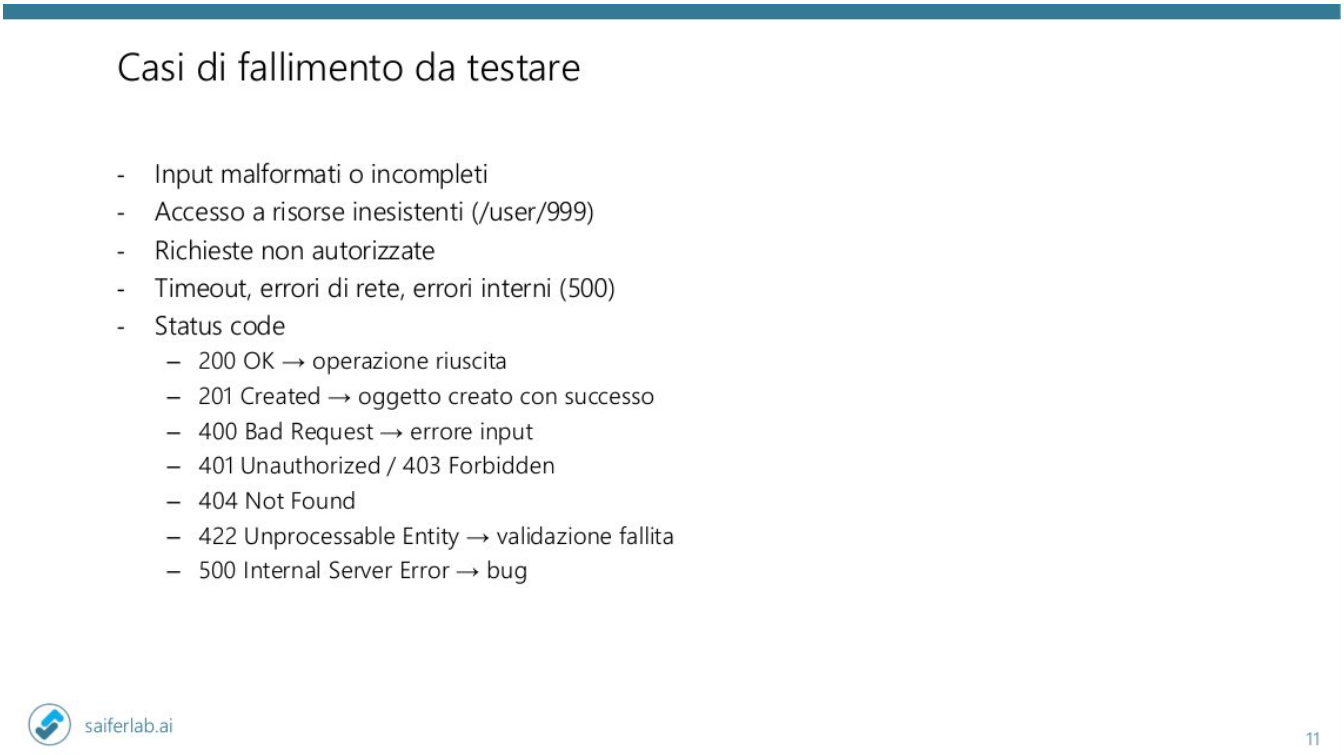
Spiegazione per studiare

Per un endpoint bisogna testare sia il percorso positivo sia i fallimenti. Gli esempi includono dati malformati, risorse inesistenti, richieste senza permessi e problemi interni; gli status 200, 201, 400, 401, 403, 404, 422 e 500 aiutano a classificare i risultati.

Definizioni, passaggi ed esempi presenti nella slide

- Input malformati o incompleti
- Accesso a risorse inesistenti (/user/999)
- Richieste non autorizzate
- Timeout, errori di rete, errori interni (500)
- Status code
- 200 OK →operazione riuscita
- 201 Created →oggetto creato con successo
- 400 Bad Request →errore input
- 401 Unauthorized / 403 Forbidden
- 404 Not Found
- 422 Unprocessable Entity →validazione fallita
- 500 Internal Server Error →bug

SLIDE ORIGINALE · diagrammi, tabelle e codice



10 / SLIDE 012 Test con FastAPI

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 12 di 14

Spiegazione per studiare

Il `TestClient` di FastAPI permette di inviare richieste simulate all'applicazione senza avviare un browser. Il test esegue una GET e verifica lo status code; il piccolo server di esempio restituisce un oggetto JSON sulla rotta principale.

Definizioni, passaggi ed esempi presenti nella slide

FastAPI supporta test con TestClient (simulazione richieste HTTP)

```
from fastapi import FastAPI

app = FastAPI()
@app.get("/")
def read_root():

return {"message": "Hello World"}

from fastapi.testclient import TestClient
from server import app
```

client = TestClient(app)

```
def test_read_root():

response = client.get("/")
assert response.status_code == 200
```

SLIDE ORIGINALE · diagrammi, tabelle e codice

Test con FastAPI

FastAPI supporta test con TestClient (simulazione richieste HTTP)

```
from fastapi import FastAPI

app = FastAPI()
@app.get("/")
def read_root():
    return {"message": "Hello World"}
```

```
from fastapi.testclient import TestClient
from server import app

client = TestClient(app)

def test_read_root():
    response = client.get("/")
    assert response.status_code == 200
```

saiferlab.ai

12

10 / SLIDE 013 Test di validazione degli input

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 13 di 14

Spiegazione per studiare

Le funzioni di validazione del framework vanno comunque verificate mediante casi corretti e scorretti. Un test dovrebbe controllare che input validi siano accettati e che quelli incompatibili con lo schema producano una risposta di errore appropriata.

Definizioni, passaggi ed esempi presenti nella slide

FastAPI gestisce automaticamente la validazione degli input, ma si può verificare che siano effettivamente gestiti correttamente In caso di errore: risposta 422 Unprocessable Entity

```
class Item(BaseModel):
```

name: str = Field(min_length=3)

price: float = Field(gt=0)

```
@app.post("/items")
def post_item(item: Item):
return {"item received": item}
def test_create_item_valid():
```

response = client.post("/items/", json={"name": "Book", "price": 5})

```
assert response.status_code == 200
def test_create_item_invalid():
```

response = client.post("/items/", json={"name": "A", "price": -5})

SLIDE ORIGINALE · diagrammi, tabelle e codice



10 / SLIDE 014 Best practice di testing API

Lezione: Testing del software e delle API

Fonte: 10_testing.pdf, pagina 14 di 14

Spiegazione per studiare

Le buone pratiche richiedono di coprire successi, input errati, stati non consentiti e condizioni di sicurezza. I test devono essere leggibili, ripetibili e coerenti con la specifica degli endpoint, così da individuare regressioni a ogni modifica del servizio.

Definizioni, passaggi ed esempi presenti nella slide

Testare tutti i percorsi possibili:

- Successo
- Fallimento per input sbagliato
- Fallimento per stato dell'app (es. utente non esiste)

Usare test automatici con TestClient

- status code
- contenuto JSON
- headers (es. token, location)

Se c'è autenticazione, bisogna verificare anche che:

- Un utente non autenticato non possa accedere
- Un utente autenticato possa solo accedere alle le sue risorse

Repository GitHub relativo a testing tutorial

SLIDE ORIGINALE · diagrammi, tabelle e codice

